Name – N.Rohith **Roll no - 2520090191**

Practical week 12

1.Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

Code:

```
  UW PICO 5.09                          File: 120.c                          Modified

    // Create producer and consumer
    pthread_create(&producer_thread,
                   NULL, producer, NULL);

    pthread_create(&consumer_thread,
                   NULL, consumer, NULL);

    // Wait for both threads
    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);

    double time_taken =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1000000000.0;

    double throughput = ITEMS / time_taken;

    printf("Buffer Size : %d\n", buffer_size);
    printf("Time        : %.6f seconds\n", time_taken);
    printf("Throughput  : %.2f items/second\n", throughput);
    printf("-----------------------------------\n");

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    free(buffer);
}

int main()
{
    printf("=======================================\n");
    printf(" PRODUCER-CONSUMER USING SEMAPHORES\n");
    printf("=======================================\n\n");

    run_test(1);
    run_test(5);
    run_test(10);
    run_test(50);
    run_test(100);

    return 0;
}

^G Get Help  ^O WriteOut  ^R Read File  ^Y Prev Pg  ^K Cut Text  ^C Cur Pos
^X Exit      ^J Justify   ^W Where is   ^V Next Pg  ^U UnCut Text ^T To Spell
```

```
nrk@nallas-mac-Book-Air ~ % gcc 120.c -o 120
120.c:84:5: warning: 'sem_init' is deprecated [-Wdeprecated-declarations]
   84 |     sem_init(&empty, 0, buffer_size);
      |     ^
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/semaphore.h:55:42: note: 'sem_init' has been explicitly marked deprecated here
   55 | int sem_init(sem_t *, int, unsigned int) __deprecated;
      |                                          ^
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/cdefs.h:223:40: note: expanded from macro '__deprecated'
  223 | #define __deprecated    __attribute__((__deprecated__))
      |                                        ^
120.c:85:5: warning: 'sem_init' is deprecated [-Wdeprecated-declarations]
   85 |     sem_init(&full, 0, 0);
      |     ^
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/semaphore.h:55:42: note: 'sem_init' has been explicitly marked deprecated here
   55 | int sem_init(sem_t *, int, unsigned int) __deprecated;
      |                                          ^
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/cdefs.h:223:40: note: expanded from macro '__deprecated'
  223 | #define __deprecated    __attribute__((__deprecated__))
      |                                        ^
120.c:115:5: warning: 'sem_destroy' is deprecated [-Wdeprecated-declarations]
  115 |     sem_destroy(&empty);
      |     ^
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/semaphore.h:53:26: note: 'sem_destroy' has been explicitly marked deprecated here
   53 | int sem_destroy(sem_t *) __deprecated;
      |                          ^
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/cdefs.h:223:40: note: expanded from macro '__deprecated'
  223 | #define __deprecated    __attribute__((__deprecated__))
      |                                        ^
120.c:116:5: warning: 'sem_destroy' is deprecated [-Wdeprecated-declarations]
  116 |     sem_destroy(&full);
      |     ^
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/semaphore.h:53:26: note: 'sem_destroy' has been explicitly marked deprecated here
   53 | int sem_destroy(sem_t *) __deprecated;
      |                          ^
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/cdefs.h:223:40: note: expanded from macro '__deprecated'
  223 | #define __deprecated    __attribute__((__deprecated__))
      |                                        ^
4 warnings generated.
nrk@nallas-mac-Book-Air ~ % ./120
=======================================
 PRODUCER-CONSUMER USING SEMAPHORES
=======================================

Buffer Size : 1
Time        : 0.060627 seconds
Throughput  : 1649430.12 items/second
-----------------------------------
Buffer Size : 5
Time        : 0.060024 seconds
Throughput  : 1666000.27 items/second
-----------------------------------
Buffer Size : 10
Time        : 0.057288 seconds
Throughput  : 1745566.26 items/second
-----------------------------------
Buffer Size : 50
Time        : 0.057356 seconds
Throughput  : 1743496.76 items/second
-----------------------------------
Buffer Size : 100
Time        : 0.056282 seconds
Throughput  : 1776766.99 items/second
-----------------------------------
nrk@nallas-mac-Book-Air ~ %
```

2.Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering

Code:



```
  UW PICO 5.09                        File: 130.c                        Modified

    pthread_create(&t1, NULL, thread1_deadlock, NULL);
    pthread_create(&t2, NULL, thread2_deadlock, NULL);

    sleep(3);

    printf("\nDeadlock scenario demonstrated.\n");

    /*
       NOTE:
       The above threads may remain blocked.
       We now demonstrate prevention separately.
    */

    printf("\n===========================================\n");
    printf(" DEADLOCK PREVENTION\n");
    printf(" Resource Ordering Strategy\n");
    printf("===========================================\n\n");

    /*
       Destroy and recreate mutexes for the
       prevention demonstration.
    */

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1_ordered, NULL);
    pthread_create(&t2, NULL, thread2_ordered, NULL);

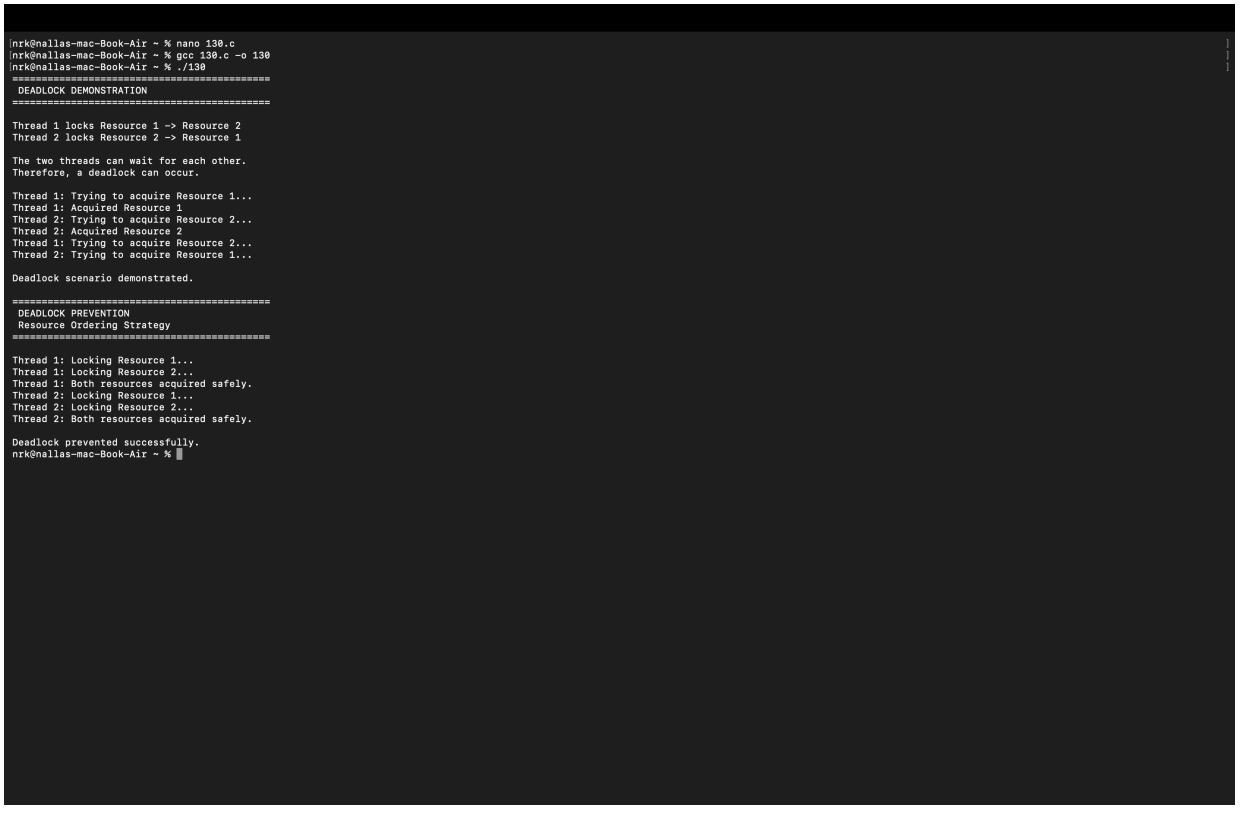
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("\nDeadlock prevented successfully.\n");

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    return 0;
}

^G Get Help  ^O WriteOut  ^R Read File  ^Y Prev Pg  ^K Cut Text  ^C Cur Pos
^X Exit      ^J Justify   ^W Where is   ^V Next Pg  ^U UnCut Text ^T To Spell
```



```
nrk@nallas-mac-Book-Air ~ % nano 130.c
nrk@nallas-mac-Book-Air ~ % gcc 130.c -o 130
nrk@nallas-mac-Book-Air ~ % ./130
===========================================
 DEADLOCK DEMONSTRATION
===========================================

Thread 1 locks Resource 1 -> Resource 2
Thread 2 locks Resource 2 -> Resource 1

The two threads can wait for each other.
Therefore, a deadlock can occur.

Thread 1: Trying to acquire Resource 1...
Thread 1: Acquired Resource 1
Thread 2: Trying to acquire Resource 2...
Thread 2: Acquired Resource 2
Thread 1: Trying to acquire Resource 2...
Thread 2: Trying to acquire Resource 1...

Deadlock scenario demonstrated.

===========================================
 DEADLOCK PREVENTION
 Resource Ordering Strategy
===========================================

Thread 1: Locking Resource 1...
Thread 1: Locking Resource 2...
Thread 1: Both resources acquired safely.
Thread 2: Locking Resource 1...
Thread 2: Locking Resource 2...
Thread 2: Both resources acquired safely.

Deadlock prevented successfully.
nrk@nallas-mac-Book-Air ~ %
```